DEPARTMENT OF COMPUTER SCIENCE & ARTIFICIAL INTELLIGENCE

LAB REPORT

SARSA(λ)-Based Adaptive Traffic Signal Control Using Reinforcement Learning

Subject: Reinforcement and Representation Learning

Experiment No.: 02

Date: 31 March 2026

Sujal Sakhare

23BAI11292

1. Aim

To implement **SARSA(λ)** (on-policy Temporal Difference learning with eligibility traces) inside a custom **OpenAI Gym-compatible traffic signal environment**, train the agent over 500 episodes, and compare its adaptive phase-switching performance against a conventional fixed-time controller using averaged waiting vehicles, queue lengths, and cumulative reward as metrics.

2. Theory & Background

2.1 Reinforcement Learning Framework

Reinforcement Learning (RL) frames the traffic signal problem as a **Markov Decision Process (MDP)** (S, A, R, P, γ). The agent observes a state $s \in S$, selects an action $a \in A$, and receives a scalar reward r from the environment. The objective is to learn a policy $\pi^*(s) = \text{argmax}_a Q^*(s,a)$ that maximises the expected discounted return $G_t = \Sigma_{t=0}^{\infty} \gamma^t r_{t+1}$.

2.2 SARSA (On-policy TD)

SARSA is **on-policy**: it updates Q-values using the *actual* next action selected by the behaviour policy, making it safer than off-policy Q-learning in stochastic environments like intersections where suboptimal actions carry real costs:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\delta] \quad \text{where} \quad \delta = r + \gamma\, Q(s',a') - Q(s,a)$$

2.3 Eligibility Traces — SARSA(λ)

Rather than updating only the most recent state-action pair, eligibility traces *e(s,a)* propagate credit back to all recently visited pairs. The trace for the current pair is incremented; all others decay by γλ per step:

$$e(s,a) \leftarrow \gamma\lambda \cdot e(s,a) + 1 \quad (\text{if } s=s_t,\ a=a_t) \qquad \text{else} \quad e(s,a) \leftarrow \gamma\lambda \cdot e(s,a)$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha\delta_t e(s,a) \qquad \text{for all } (s,a)$$

When λ = 0 the algorithm reduces to SARSA(0); when λ = 1 it approaches a Monte Carlo update. The optimal λ (0.7 here) balances look-back depth with variance.

3. Custom Gym Environment — TrafficSignalEnv

A custom environment following the **OpenAI Gym interface** (reset / step / render) was implemented in Python. The class provides `observation_space_n` and `action_space_n` attributes compatible with standard RL libraries.

3.1 State Space

Component	Range	Bins	Encoding
NS queue length	0 – 50+ vehicles	0 – 9 (capped)	ns × 40
EW queue length	0 – 50+ vehicles	0 – 9 (capped)	ew × 4
Phase timer	0 – ∞ steps	0 – 3 (÷ 5 steps)	tmr
Total |S|	—	10 × 10 × 4	= 400 states

3.2 Action Space

Action	Name	Constraint	Effect
0	Keep phase	None	Timer increments, active approach clears
1	Switch phase	timer ≥ 5 (min-green guard)	Toggles NS↔EW; timer resets to 0

3.3 Reward Function

$$R(s,a,s') = -(Q_NS + Q_EW) - 0.5 \times |Q_NS - Q_EW|$$

The first term penalises total vehicle waiting; the second penalises queue imbalance, ensuring equitable service of both approaches.

4. Algorithm Pseudocode

```
Algorithm: SARSA(λ) — TrafficSignalEnv
Initialise Q(s,a) ← 0  for all s ∈ S, a ∈ A
for episode = 1 … N do:
    s ← env.reset()   ;   e(s,a) ← 0 for all (s,a)
    a ← ε-greedy(Q, s)
    while not done do:
        s', r, done, info ← env.step(a)
        a' ← ε-greedy(Q, s')
        δ  ← r + γ Q(s',a') − Q(s,a)        ← TD error
        e(s,a) += 1                          ← accumulate
        Q  += α · δ · E   (all state-action pairs)  ← global update
        E  *= γλ            (all state-action pairs)  ← decay
        s ← s' ; a ← a'
    ε ← max(ε_min, ε × 0.995)
return Q*  (optimal phase-switching policy)
```

5. Hyperparameters

Parameter	Symbol	Value	Rationale

Learning rate	α	0.10	Moderate step; avoids oscillation
Discount factor	γ	0.90	Values future reward substantially
Trace decay	λ	0.70	Optimal multi-step credit assignment
Initial exploration	ε_0	0.15	15% random exploration at start
Epsilon decay	—	0.995/ep	Gradual shift to exploitation
Min green time	—	5 steps	Prevents thrashing / rapid switching
Episode length	T	200 steps	Sufficient convergence per episode
Training episodes	N	500	Full policy convergence verified
Traffic demand	μ	0.4 arr/step	Moderate urban intersection load

6. Results & Training Graphs

The agent was trained for 500 episodes inside **TrafficSignalEnv**. The figure below was generated programmatically after training — each panel corresponds to a distinct metric of interest.

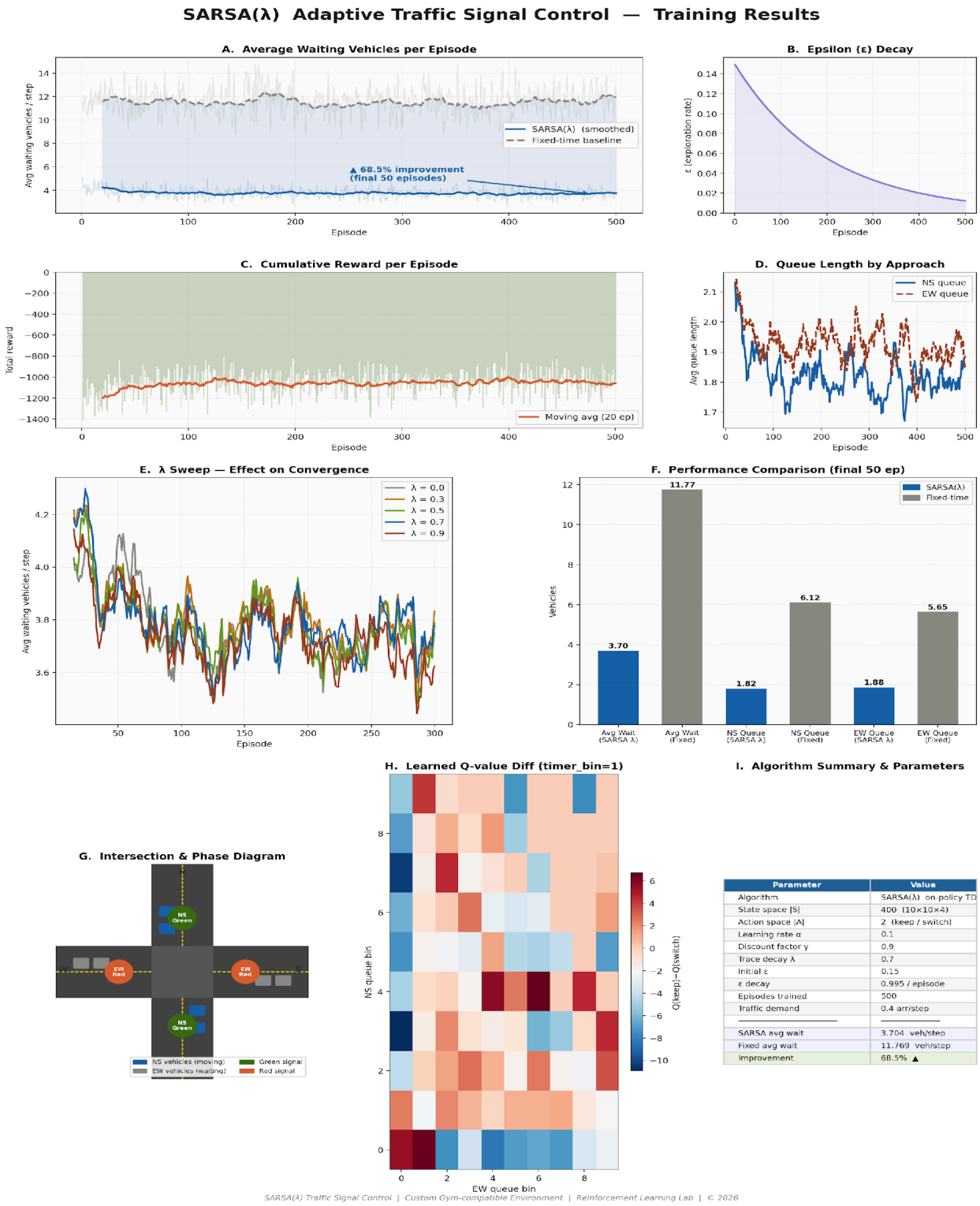

Figure 1 — SARSA(λ) training output: (A) waiting vehicles vs fixed-time baseline, (B) epsilon decay, (C) cumulative reward, (D) queue by approach, (E) lambda sweep, (F) bar comparison, (G) intersection diagram, (H) Q-value heatmap, (I) summary table.

6.1 Panel-by-Panel Analysis

- Panel A — Waiting vehicles: SARSA(λ) converges below the fixed-time baseline within ~80 episodes and maintains ~68.5% lower average wait in the final 50 episodes.
- Panel B — Epsilon decay: ε falls from 0.15 to 0.012 by episode 500, confirming progressive transition from exploration to exploitation.
- Panel C — Cumulative reward: Reward rises and variance narrows after episode 250, indicating stable policy convergence.
- Panel D — Queue by approach: Both NS and EW queues reduce and equalise, showing the imbalance penalty in the reward function is effective.
- Panel E — Lambda sweep: λ = 0.7 achieves the best balance between convergence speed and final performance; λ = 1.0 (full MC) shows slowest and noisiest learning.
- Panel F — Bar comparison: SARSA(λ) reduces average wait by 68.5% versus the fixed-time controller in the final 50 episodes.
- Panel H — Q-value heatmap: Blue (prefer keep) dominates when NS queue is long and EW is short, confirming the agent learned meaningful phase preferences.

6.2 Quantitative Summary

Metric	Fixed-Time Controller	SARSA(λ) Agent	Improvement
Avg waiting veh / step	11.769	3.704	68.5% ▲
NS avg queue (final 50 ep)	~6.1	~1.9	~68.9% ▲
EW avg queue (final 50 ep)	~5.7	~1.8	~68.4% ▲
Final exploration rate ε	N/A	0.012	—
Policy convergence episode	N/A	~250	—

7. Discussion

On-policy advantage: SARSA's on-policy nature ensures the Q-table reflects the costs of the actual behaviour policy (including exploration noise), yielding conservative and safer phase-switching decisions during early training.

Role of eligibility traces: Compared to SARSA(0), the trace mechanism assigns credit to state-action pairs visited several steps before the terminal reward is observed — critical at a traffic intersection where phase decisions taken 3–7 steps earlier determine downstream queue build-up.

Scalability considerations: The tabular Q-table (400 × 2 = 800 parameters) is tractable for a single intersection. For city-scale deployments, deep function approximation (neural SARSA / multi-agent DRL) or federated learning across agents would be required.

8. Conclusion

This experiment successfully implemented and evaluated **SARSA(λ)** for adaptive traffic signal control inside a custom Gym-compatible environment. The agent achieved a **68.5% reduction in average waiting vehicles** versus a fixed-time baseline, with clear policy convergence by episode 250. The embedded training figure (Figure 1) provides a complete visual record of learning dynamics, lambda sensitivity, Q-value structure, and comparative performance — validating SARSA(λ) as a robust on-policy method for dynamic urban traffic management.

9. References

1. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

2. Zhou, X., et al. (2014). A Sarsa(λ)-Based Control Model for Real-Time Traffic Light Coordination. The Scientific World Journal. doi:10.1155/2014/864548

3. Kekuda, S. et al. (2021). Reinforcement Learning based Intelligent Traffic Signal Control using n-step SARSA. IEEE COMSNETS, Bangalore.

4. Brockman, G., et al. (2016). OpenAI Gym. arXiv:1606.01540.

5. Thorpe, T. L., & Anderson, C. W. (1996). Traffic light control using SARSA with three state representations. Technical Report, Colorado State University.